

DEPENDENCY ENGINE

Architecture Design Document
MVP Version

Architect By : Eranda Udeshika Horanagama
Author : Eranda Udeshika Horanagama

DEPENDENCY ENGINE	1
1. Executive Summary	2
2. Architecture Design/Diagrams	2
3. POC Implementation	3
Source code:	3
How to run	3
Component Mapping Table	4
3. Design Principles	6
4. Final Thoughts	7
Key Assumptions	7
Cross-Cutting Concerns	7
Architectural Tradeoffs	7
Improvements	7
5. Appendix	8
CICD Build Event Payload (Used for Cache Eviction) e.g.	8

1. Executive Summary

This document outlines the high-level hybrid architecture for an Edge-Accelerated Dependency Engine designed to provide real-time code intelligence to AI Copilots under a strict sub-second response SLA. The system bridges centralized cloud-native graph data (AWS Neptune) with real-time, uncommitted local workspace edits, eliminating network latency.

Note: This represents the MVP (Minimum Viable Product) version, focusing on core edge cache merging (Shadow Masking), kernel-level local file watching, and optimized Markdown serialization over the Model Context Protocol (MCP).

Production-grade cloud infrastructure automation and complex data migration pipelines are considered out of scope for this assignment. The AWS stack was intentionally incorporated to better visualize cloud-to-edge integration

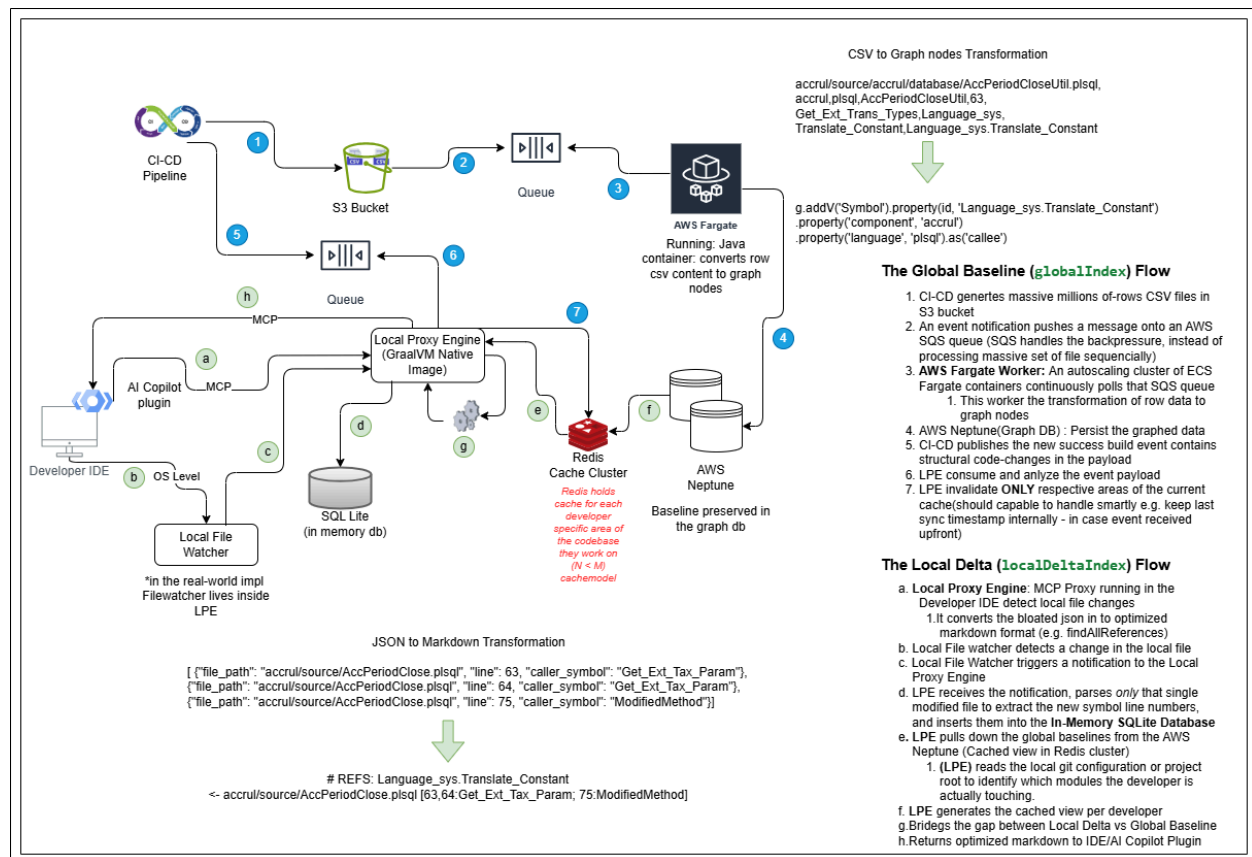
2. Architecture Design/Diagrams

Key Idea in the Design

The Global Baseline (globalIndex**):** remains completely untouched and read-only - used cached subsets of it for operationl efficiency (Part-1 Alignment in the assignment).

The Local Delta (localDeltaIndex**):** remains isolated in the developer memory layer (Part-2 Alignment in the assignment).

The Merge (combinedResults**):** is transient. It is calculated dynamically in RAM in less than a millisecond *only when requested*, and then thrown away after it is sent to the AI Copilot via MCP.



*The AWS stack was intentionally incorporated to better visualize cloud-to-edge integration

Diagram source code :

https://github.com/Erandauh/Dependency-Engine-POC/blob/main/docs/Dependency_Engine_Arch.drawio

Full sized PNG Image:

https://github.com/Erandauh/Dependency-Engine-POC/blob/main/docs/Dependency_Engine_Arch.drawio

3. POC Implementation

Source code:

<https://github.com/Erandauh/Dependency-Engine-POC/tree/main>

How to run

Providing the guidance align to the assignment request itself

Step 1:

- Ingestion of symbol and dependency file (example provided by us) to build the dependency “index”

- This happens in the first step of the sequence, once you run

```
DependencyEnginePoC.main
```

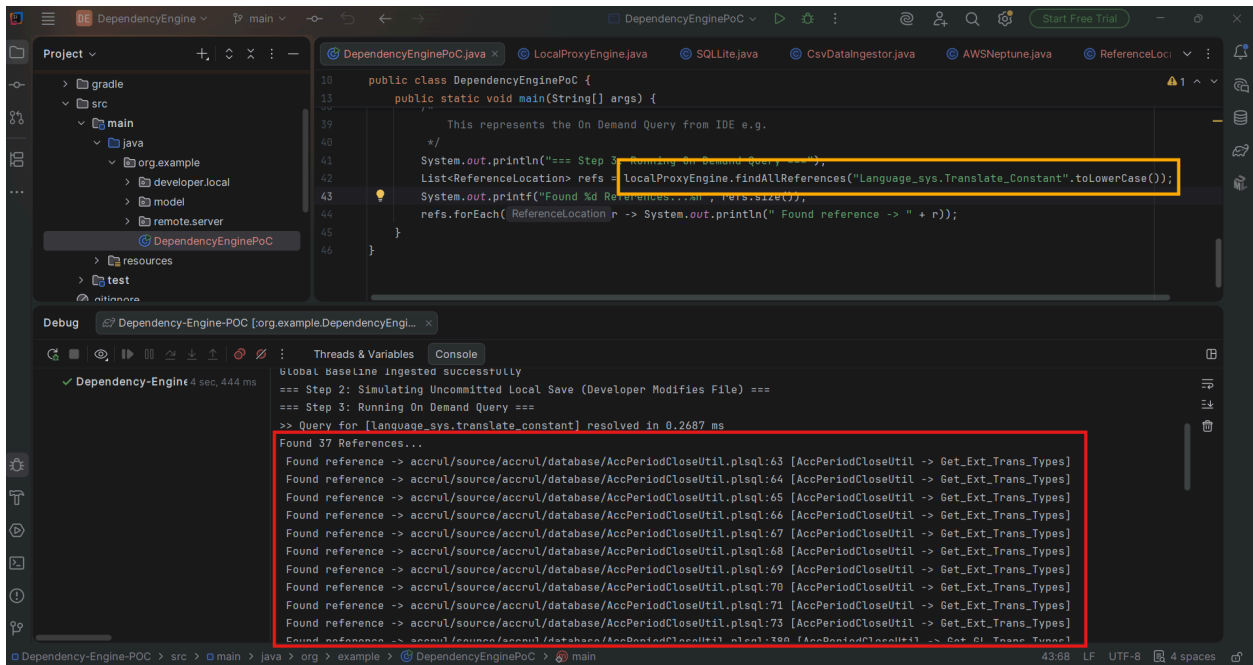
Step 2:

- Inference call to look up dependencies for any symbol

- This is currently hardcoded in `DependencyEnginePoC.main` L#43 “step 3”

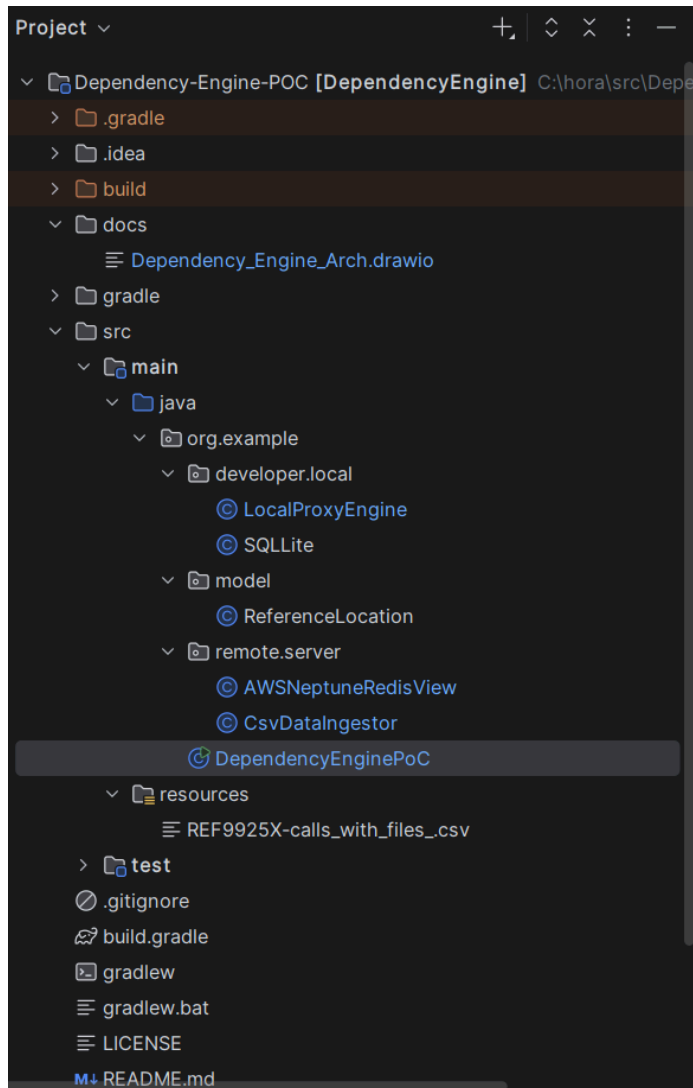
```
localProxyEngine.findAllReferences("Language_sys.Translate_Constant".toLowerCase()  
e());
```

Once you run the code, output would be as following:



Component Mapping Table

Hope following would help you to understand which components in the current Java mimics the behavior of which architechural component



File Name	Architectural Role	Structural Engine Representation
org.example.model. ReferenceLocation.java	Domain Data Model	An immutable data carrier representing a single dependency coordinate.
org.example.remote.server. AWSNeptuneRedisView.java (Uses a thread-safe ConcurrentHashMap keyed by	Cached subset of the Global Baseline Data	Simulates AWS Neptune cloud graph Cached subset

referenced_symbol to guarantee instant O(1) lookups)		
org.example.remote.server. CsvDataIngestor.java	Global Data Ingestion Pipeline	Simulates the streaming csv data to graph db (Step e). It parses the raw data file (REF9925X...csv) from the resources folder line-by-line to graph db(AWS Neptune).
org.example.developer.local. SQLite.java	Developer Local Delta	Simulates the transient data representing the local workspace
org.example.developer.local. LocalProxyEngine.java maintains a synchronized tracking set (maskedLocalFiles) to run shadow masking at runtime - dropping obsolete global baseline data for modified files and overlaying the fresh local updates on the fly.	Brian Engine	The core orchestrator of Step f. (Local Proxy Engine)
org.example. DependencyEnginePoC.java	E2E Simulation Harness	The executable entry point containing the main method. It runs the entire proof of concept sequence

3. Design Principles

- **Separation of Concerns**

Enforces a strict boundary between the cloud ecosystem (remote.server) and the developer workspace client (developer.local). Cloud handles heavy graph processing (Neptune/Fargate); Local isolates workspace edits and AI tools (MCP/SQLite).

- **State Immutability**

The global baseline stays a clean, read-only Source of Truth. Local workspace tracking remains completely lightweight via cached

- **Semantic Compression (Optimized formats)**

Intercepts verbose database graph structures (JSON payloads) and transform them into a Markdown before sending them over the MCP pipeline.

4. Final Thoughts

Key Assumptions

- Assumes codebase graph snapshots generated via the CI/CD pipeline do not shift structurally between build cycles.
- Assumes the developer's machine runs a pre-installed, high-performance runtime environment (e.g., GraalVM Native Image) capable of executing the Local Proxy Engine binary natively.

Cross-Cutting Concerns

- **Code Security:** Code snippets and structural changes stay securely on the developer's laptop; only metadata hashes and tracking coordinates travel across the network boundary.
- Thread safety is guaranteed during simultaneous file changes by leveraging thread-safe data structures (`ConcurrentHashMap` and `CopyOnWriteArrayList`).

Architectural Tradeoffs

- **Selective Domain Caching:** Instead of mirroring the entire multi-gigabyte global repository graph onto the laptop, the LPE intercepts the workspace scope to cache *only* the developer's active components (e.g., `accru1`). We sacrifice **local cross-domain impact analysis** and accept occasional **cache-miss network penalties** when a developer jumps to a new module. In return, we protect the developer's laptop from memory exhaustion and keep the edge footprint tightly optimized.

Improvements

- **Sync only the Delta:** Currently, the architecture implies downloading a massive baseline CSV from S3 via the proxy engine to hydrate the Baseline. Even though caching is in place to optimize the flow. If we can move from a bulk file download model

to an incremental delta sync model. The LPE can poll a lightweight backend endpoint that only sends graph updates (additions/deletions) that occurred since the developer's last sync timestamp...

5. Appendix

CICD Build Event Payload (Used for Cache Eviction) e.g.

```
{ "event": "BUILD_SUCCESS",  
  "version_hash": "v2026.06.07-1105",  
  "timestamp": "2026-06-07T11:05:00Z",  
  "build_scopes":  
  { "components": ["accrul", "enterp"],  
    "logical_units": ["AccPeriodClose", "CompanyAddress"],  
    "files": [ "accrul/source/accrul/database/AccPeriodCloseUtil.plsql" ]  
  }  
}
```